

Data-Driven Investment Intelligence Using NIFTY-50 Market Data

TECHNICAL REPORT

TANVI MANDAN

24118072

tanvi_m@mt.iitr.ac.in

Table of Contents

- 1. Executive Summary & Pipeline Architecture 3
- 2. Exploratory Data Analysis (EDA) & Dataset Properties 5
- 3. Data Preprocessing & Vectorized Feature Engineering 7
- 4. Machine Learning Forecast Model Architecture 9
- 5. Risk Assessment Framework & Mathematical Formulations10
- 6. Modern Portfolio Optimization & SLSQP Constrained Solver11
- 7. Explainable AI (XAI) & Dynamic Signal Generation 12
- 8. Platform Implementation Insights & Engineering Lessons13

1. Executive Summary & Pipeline Architecture

Executive Summary

This technical report details the mathematical models, algorithmic choices, data structures, and engineering implementations underpinning the NIFTY-50 AI-Powered Investment Intelligence Platform. The system has been engineered to ingest daily historical stock data (prices, volumes, turnovers) for NSE blue-chip equities and transform them into quantitative decision-support insights.

Crucially, the system runs locally and supports isolated cloud environments (such as Google Colab) using a standard schema matched to public historical datasets (specifically, Rohan Rao's NIFTY-50 dataset from 2000-2021). This platform bridges the gap between raw data processing and live advisory by compiling a high-performance predictive engine, portfolio optimizer, risk assessor, and model explainer into a responsive Streamlit dashboard.

System & Data Pipeline Architecture

The system is designed around a strictly decoupled, modular architecture. It segregates logic into feature calculations, statistical evaluations, machine learning models, and optimization solvers. This prevents script bloat and ensures the mathematical formulas can be verified independently via automated tests.

- **Data Loader Component:** Checks file system; loads stock CSVs & metadata; handles delistings & imputation.
- **Feature Engineering Engine:** Computes rolling technical indicators (Trend, Momentum, Volatility, Volume).
- **Machine Learning Models:** HistGradientBoosting models estimate 5-day forward return and direction.
- **Risk Assessment Engine:** Computes annualized return/volatility, Sharpe, Sortino, Max Drawdown, Beta.
- **Portfolio Construction Solver:** Employs SciPy's SLSQP solver for Conservative, Balanced, and Aggressive asset mixes.
- **Explainability & Advisor:** Permutation feature importances + narrative explanations + personalized capital allocation.

Modularity & Decoupling Benefits

By avoiding direct integration of data loading and view rendering, the pipeline achieves:

- **Testability:** A verification suite (`src/test_suite.py`) validates each equation without launching a server.
- **Concurrency:** Read operations on model files are separated from write operations (model training), avoiding lock-ups.
- **Portability:** The same core code runs in a terminal, a Streamlit dashboard, or a Google Colab notebook.

2. Exploratory Data Analysis & Dataset Properties

The system's database comprises daily historical price series (Open, High, Low, Close, Last, VWAP) and volume information for the constituent stocks of the NIFTY-50 index. The data spans a 21-year historical window from January 3, 2000, to April 30, 2021. This historical frame contains diverse market regimes, including the 2008 Global Financial Crisis, the 2013 taper tantrum, and the 2020 COVID-19 crash, providing a robust training corpus.

Data Attributes & Statistical Profiles

Financial daily returns are heavily non-normal, characterized by leptokurtosis (fat tails) and volatility clustering. Modeling these returns requires robust algorithms that do not assume normal return distributions. Below is an overview of the statistical properties of key sectors represented in the index:

Sector/Industry	Representative Stock	Volatility Profile	Avg. Annual Return	Risk Level
FINANCIAL SERVICES	HDFCBANK / ICICIBANK	Moderate-High	14% - 18%	Moderate
IT	TCS / INFY	Moderate	16% - 20%	Moderate
ENERGY	RELIANCE / BPCL	Moderate-High	12% - 16%	Moderate-High
CONSUMER GOODS	HINDUNILVR / ITC	Low-Moderate	10% - 14%	Low
PHARMA	SUNPHARMA / CIPLA	Moderate	11% - 15%	Low-Moderate

Asset Cross-Correlations

Cross-asset correlation analysis shows that sector peers (e.g. INFY and TCS, or HDFCBANK and ICICIBANK) exhibit strong positive correlations (frequently >0.70). However, defensive consumer stocks (like HINDUNILVR) and IT services display lower correlation relative to cyclical banking stocks. The

portfolio construction engine leverages these non-uniform correlation vectors to run Mean-Variance Optimization, allocating capital across uncorrelated assets to reduce systemic portfolio volatility.

3. Preprocessing & Technical Feature Engineering

Financial time series are notoriously noisy. To extract predictive signals, the raw prices are processed through a vectorized feature calculation module (src/features.py). This module implements rolling windows to generate three core families of indicators: Trend, Momentum, and Volatility.

Vectorized Formulation of Key Indicators

Indicator	Mathematical Formula / Definition
Simple Moving Average (SMA)	$SMA_t = (1/N) * \sum Close_{\{t-i\}} \text{ (for } i=0 \text{ to } N-1)$ Used for 10-day, 50-day, and 200-day rolling windows.
Exponential Moving Average (EMA)	$EMA_t = Close_t * \alpha + EMA_{\{t-1\}} * (1 - \alpha)$ Where weight $\alpha = 2 / (N + 1)$. Used for 12, 26, and 50-day windows.
Relative Strength Index (RSI)	$RSI = 100 - [100 / (1 + RS)]$ Where $RS = (Avg. Gain / Avg. Loss)$ over a 14-day rolling window.
MACD Histogram	MACD Line = $EMA(Close, 12) - EMA(Close, 26)$ Signal Line = $EMA(MACD \text{ Line}, 9)$ Histogram = MACD Line - Signal Line
Bollinger Bands	Middle Band = $SMA(Close, 20)$ Upper/Lower Bands = $Middle \text{ Band} \pm (2 * \sigma_{20})$ Where σ_{20} is the 20-day standard deviation of Close prices.

Volatility, Momentum & ML Feature Formatting

In addition to rolling price-based technical indicators, the feature engineering engine calculates the following signals to feed into the prediction models:

- **Annualized Historical Volatility:** Rolling 20-day standard deviation of daily log returns, scaled to business-day annual terms: $\sigma_{\text{ann}} = \sigma_{\text{20d}} * \sqrt{252}$
- **Price Momentum:** Rolling 5-day Rate of Change (ROC) and SMA ratios.
- **Volume Indicators:** 5-day rolling average volume relative to the 20-day average volume to detect sudden market participation surges.

Feature Normalization

To prevent gradient boosting models from learning nominal price levels (which would fail to generalize on trending time series), all nominal price-based technical indicators are divided by the latest Close price. This scales the values to be relative ratios. For instance, the EMA is formatted as: $\text{EMA_Scaled} = \text{EMA} / \text{Close}$.

Target Variable Definitions

For model training, the target labels are shifted to represent forward-looking values (avoiding look-ahead data leakage):

1. Regression Target (Target_Return): The actual log return over the next 5 business days:

$$\text{Target_Return}_t = \ln(\text{Close}_{\{t+5\}} / \text{Close}_t)$$

2. Classification Target (Target_Direction): Binary indicator representing whether the 5-day forward return is positive:

$$\text{Target_Direction}_t = 1 \text{ if } \text{Target_Return}_t > 0 \text{ else } 0$$

4. Machine Learning Model Architecture

The Stock Predictor Engine (src/predictor.py) implements a dual-model forecasting strategy: a regression model predicts the magnitude of the forward 5-day returns, and a classification model predicts the binary direction (UP vs DOWN) with probability confidence scores.

Algorithm Selection: HistGradient Boosting

The platform utilizes scikit-learn's HistGradientBoostingRegressor and HistGradientBoostingClassifier. These algorithms are selected because:

- Native Missing Value Handling: They handle missing feature cells efficiently without requiring imputation heuristics.
- Outlier Robustness: Boosting trees partition data using numerical threshold boundaries, neutralizing extreme return spikes.
- Non-Linear Mappings: They capture complex technical indicator interaction patterns (e.g. overbought RSI combined with a Bollinger band breach).
- Execution Speed: Binning numerical features into integer-valued bins accelerates training by orders of magnitude.

Time-Series Validation Strategy

Because stock market data is non-stationary, standard k-fold random split validation causes severe look-ahead bias (data leakage). To enforce temporal ordering, the dataset is split chronologically: the first 80% of historical data is allocated for training, and the subsequent 20% is held back strictly for testing model performance.

Model Type	Base Estimator	Max Iterations	Validation Split	Random Seed
Regressor	HistGradientBo ostingRegressor	100 Trees	Chronological (80/20)	Seed = 42
Classifier	HistGradientBo ostingClassifier	100 Trees	Chronological (80/20)	Seed = 42

5. Risk Assessment Framework & Mathematical Formulations

Risk modeling is performed by a dedicated analytics module (src/risk.py). It provides standard risk statistics, which are displayed in the Key Performance Stats cards of the user dashboard.

Mathematical Formulas & Definitions

Risk Metric	Mathematical Formulation	Financial Description
Annualized Return	$R_{ann} = R_{daily} * 252$	The expected annual compound rate of return.
Annualized Volatility	$\sigma_{ann} = \sigma_{daily} * \sqrt{252}$	The annualized standard deviation of log returns.
Sharpe Ratio	$SR = (R_{ann} - R_f) / \sigma_{ann}$	Calculates the excess return earned per unit of total volatility.
Sortino Ratio	$Sortino = (R_{ann} - R_f) / \sigma_{down}$	Where σ_{down} is the annualized downside semi-standard deviation.
Maximum Drawdown	$MDD = \text{Min}((P_t - \text{Peak}_t) / \text{Peak}_t)$	Measures the worst peak-to-trough drop in historical asset value.
Systematic Beta (β)	$\beta = \text{Cov}(R_i, R_m) / \text{Var}(R_m)$	Measures asset volatility relative to the index return (R_m).

6. Modern Portfolio Optimization & SLSQP Constrained Solver

Capital allocation is computed dynamically based on the user's risk tolerance profile (Conservative, Balanced, or Aggressive) in `src/portfolio.py`. The platform uses SciPy's Sequential Least Squares Programming (SLSQP) optimization solver.

Mean-Variance Optimization (MVO)

The optimizer operates on the annualized sample covariance matrix (Σ) and mean returns (μ) of the loaded assets. The profiles are formulated as follows:

1. Conservative (Minimum Variance): Minimize $w^T \Sigma w$
2. Balanced (Maximum Sharpe): Maximize $(w^T \mu - R_f) / \sqrt{w^T \Sigma w}$
3. Aggressive (Momentum Tilted): Selects top 4 assets with the highest predicted return and allocates weights (40%, 30%, 20%, 10%).

Optimization Constraints

To prevent highly concentrated, unrealistic allocations, the solver implements strictly enforced constraints:

- Fully Invested Constraint: Sum of weights = 1.0 ($\sum w_i = 1.0$)
- Long-Only Constraint: Minimum allocation for any asset is 0.0 ($w_i \geq 0.0$)
- Diversification Caps: Maximum weight is capped to 40% for Conservative, 35% for Balanced.

7. Explainable AI (XAI) & Dynamic Signal Generation

A critical requirement for deploying machine learning models in finance is transparency. Black-box predictions can lead to catastrophic losses if the drivers of a decision are unknown. The platform addresses this by combining statistical feature importance with a rule-based signal explanation engine.

Permutation Feature Importance (SHAP-Proxy)

To identify which technical indicators drive predictions for each stock, the platform calculates Permutation Feature Importance (defined in `src/explainability.py`):

1. A baseline model validation score (MAE or Accuracy) is computed on the test split.
2. A single technical feature column (e.g., `RSI_14`) is randomly shuffled, breaking its relationship with the target.
3. The test set is scored again, and the decrease in model performance is recorded.
4. The process is repeated across all features, and the relative importances are plotted.

Rule-Based Technical Narratives

- RSI Signals: Flags assets as overbought ($RSI > 70$) or oversold ($RSI < 30$).
- Bollinger Bands: Identifies price levels breaking the bands, explaining volatility shifts.
- Moving Averages: Detects golden crosses (SMA 10 crossing above SMA 50).

8. Platform Implementation Insights & Engineering Lessons

During development and validation against the real Kaggle NIFTY-50 dataset, several critical bugs were fixed:

1. Resolving the 0.00 Systematic Beta Bug

The Issue: Initially, the risk module calculated Beta against a single stock (RELIANCE) as market proxy, causing Beta to hover close to 0.00.

The Fix: The market index return is now computed as the daily cross-sectional average of all constituent stocks, yielding realistic Betas.

2. Streamlit Multi-Threaded Write Collisions

The Issue: Multiple users loaded the dashboard simultaneously, causing file lock collisions and PermissionErrors on .pkl files.

The Fix: Implemented global sequential pre-training of models at startup inside @st.cache_resource block.

3. MVO Optimizer Instability in High Interest-Rate Environments

The Issue: If risk-free rate exceeded expected returns, maximizing negative Sharpe forced the optimizer to maximize volatility.

The Fix: Implemented risk-free rate proxy ($0.5 * \text{Max_Return}$) if maximum asset return falls lower than the risk-free rate.

4. Delisted Stocks & Empty CSV Handling

The Issue: Delisted empty stocks (0 rows) crashed the application during training loops at startup.

The Fix: Updated load_all_stocks() to exclude stocks with < 100 historical points and added try-except blocks.

Conclusion

The NIFTY-50 AI-Powered Investment Intelligence Platform provides a comprehensive framework for stock forecasting, risk analysis, and asset allocation. By integrating GBDT forecasting models with a Mean-Variance Optimization solver and explainable AI, the platform delivers a reliable decision-support system for NSE equities.